DATA 211
Fundamentals of Data Science

LECTURE 2: PANDAS & SCIKIT-LEARN

Why Python?

- Popular Choice
  - Widely used in data science due to simplicity and power.
- Powerful Libraries
  - Supports powerful libraries for data science.
- Great for Data Wrangling
  - Clean, reshape, and transform data with ease.
- Strong Community & Resources
  - Active forums, tutorials, and open-source tools.
- Integration & Automation
  - Easily integrates with databases, APIs, and dashboards.

Install Libraries

Individually:

- pip install seaborn

- pip install numpy

- pip install pandas

- pip install jupyter

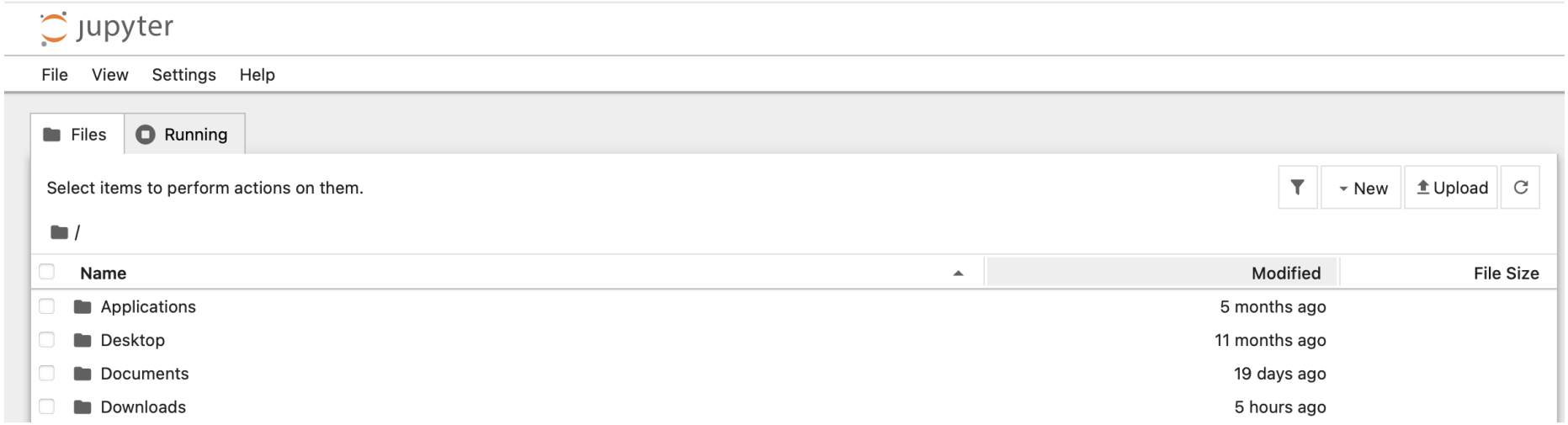
All together (common)

- **pip install numpy pandas matplotlib seaborn jupyter jupyterlab notebook**

- pip install mlxtend scipy scikit-learn

Running **Jupyter Notebook**

- **jupyter notebook**



1. Pandas

BASIC INTRODUCTION

Introduction to Pandas

- Pandas is a powerful Python library for data analysis and manipulation.
- It provides Series (1D) and DataFrame (2D) structures for handling structured data.

```
import pandas as pd
import numpy as np

# Print the Pandas version
print("Pandas version:", pd.__version__)
```

```
Pandas version: 2.2.3
```

- Key features include data cleaning, analysis tools, and integration with other
- Python libraries. Ideal for working with tabular data from various sources.

Creating a Pandas Series

- A Series is a one-dimensional labeled array that can hold any data type with an index.

```
# Create a Pandas Series
series = pd.Series(data=['Karachi', 'Lahore', 'Islamabad'],
                   index=['A', 'B', 'C'])
print("Pandas Series:")
print(series)
```

```
Pandas Series:
A      Karachi
B       Lahore
C    Islamabad
dtype: object
```

- Series are fundamental building blocks in Pandas, useful for single-column data with custom indexes.

Creating a DataFrame from Dictionary

- DataFrames are 2D tables created from dictionaries where keys become column names.

```
# Create a Pandas DataFrame from dictionary
data = {'Fruits': ['Mango', 'Apple', 'Banana', 'Orange'],
        'Quantity': [40, 20, 25, 10],
        'Price': [80, 100, 50, 70]}

df = pd.DataFrame(data)
print("\nPandas DataFrame:")
print(df)
```

```
Pandas DataFrame:
   Fruits  Quantity  Price
0   Mango        40     80
1   Apple        20    100
2  Banana        25     50
3  Orange        10     70
```

- DataFrames allow mixed data types and provide powerful data manipulation capabilities.

DataFrame Inspection Methods

- These methods help understand data types, dimensions and preview data quickly.

Method/Operation	Example Output
df.dtypes	Fruits object Quantity int64 Price int64
df.shape	(4, 3)
df.info()	<class 'pandas.core.frame.DataFrame'> RangeIndex: 4 entries, 0 to 3 Data columns (total 3 columns): # Column Non-Null Count Dtype --- ------ -------------- ----- 0 Fruits 4 non-null object 1 Quantity 4 non-null int64 2 Price 4 non-null int64
df.head(2)	Fruits Quantity Price 0 Mango 40 80 1 Apple 20 100

```
Pandas DataFrame:
   Fruits  Quantity  Price
0   Mango        40     80
1   Apple        20    100
2  Banana        25     50
3  Orange        10     70
```

DataFrame Inspection Methods

- These methods help understand data types, dimensions and preview data quickly.

Method/Operation	Example Output
df.tail(2)	Fruits Quantity Price 2 Banana 25 50 3 Orange 10 70
df.sample(2)	Fruits Quantity Price 3 Orange 10 70 1 Apple 20 100
df['Fruits']	0 Mango 1 Apple 2 Banana 3 Orange Name: Fruits, dtype: object
df[['Fruits', 'Price']]	Fruits Price 0 Mango 80 1 Apple 100 2 Banana 50 3 Orange 70

```
Pandas DataFrame:
   Fruits  Quantity  Price
0   Mango        40     80
1   Apple        20    100
2  Banana        25     50
3  Orange        10     70
```

DataFrame Inspection Methods

- These methods help understand data types, dimensions and preview data quickly.

Method/Operation	Example Output
df['Fruits'].value_counts()	Mango 1 Apple 1 Banana 1 Orange 1 Name: Fruits, dtype: int64
df['Fruits'].unique()	['Mango' 'Apple' 'Banana' 'Orange']
df.nlargest(2, 'Quantity')	Fruits Quantity Price 0 Mango 40 80 2 Banana 25 50
df.nsmallest(2, 'Quantity')	Fruits Quantity Price 3 Orange 10 70 1 Apple 20 100

Pandas DataFrame:

	Fruits	Quantity	Price
0	Mango	40	80
1	Apple	20	100
2	Banana	25	50
3	Orange	10	70

Conditional Selection and Indexing

- Filter rows based on conditions for targeted analysis.

```
# Conditional selection
print("\nRows with Price > 50:")
print(df[df.Price > 50])

# loc and iloc
print("\nUsing loc (Fruits to Price columns):")
print(df.loc[:, 'Fruits':'Price'])

print("\nUsing iloc (rows 1 to 2):")
print(df.iloc[1:3])

# at and iat
print("\nValue at row 1, Price column:", df.at[1, 'Price'])
print("Value at row 1, column 2:", df.iat[1, 2])
```

```
Rows with Price > 50:
    Fruits  Quantity  Price
0   Mango         40     80
1   Apple         20    100
3  Orange         10     70

Using loc (Fruits to Price columns):
    Fruits  Quantity  Price
0   Mango         40     80
1   Apple         20    100
2  Banana         25     50
3  Orange         10     70

Using iloc (rows 1 to 2):
    Fruits  Quantity  Price
1   Apple         20    100
2  Banana         25     50

Value at row 1, Price column: 100
Value at row 1, column 2: 100
```

Changing Data Types in a DataFrame

- Pandas allows you to explicitly convert the data types of columns in a DataFrame using the .astype() method.
- This is helpful when optimizing memory usage or preparing data for analysis.

```
# Change data types
df['Quantity'] = df['Quantity'].astype('int32')
df['Price'] = df['Price'].astype('float64')
print("\nAfter changing data types:")
print(df.dtypes)
```

```
After changing data types:
Fruits       object
Quantity      int32
Price       float64
dtype: object
```

Sorting and Reindexing

- Organize data by values and reset indexes.

```
:  # Sorting by values
   print("\nSorted by Price (ascending):")
   print(df.sort_values('Price', ascending=True))

   print("\nSorted by Price (descending):")
   print(df.sort_values('Price', ascending=False))

   # Reindexing
   df.reset_index(drop=True, inplace=True)
   print("\nAfter resetting index:")
   print(df)
```

Sorted by Price (ascending):

	Fruits	Quantity	Price
2	Banana	25	50.0
3	Orange	10	70.0
0	Mango	40	80.0
1	Apple	20	100.0

Sorted by Price (descending):

	Fruits	Quantity	Price
1	Apple	20	100.0
0	Mango	40	80.0
3	Orange	10	70.0
2	Banana	25	50.0

After resetting index:

	Fruits	Quantity	Price
0	Mango	40	80.0
1	Apple	20	100.0
2	Banana	25	50.0
3	Orange	10	70.0

Renaming Columns

- Column names can be updated using rename() method

```
# Renaming columns
df.rename(columns={'Fruits': 'FRUITS',
                   'Quantity': 'QUANTITY',
                   'Price': 'PRICE'},
          inplace=True)
print("\nAfter renaming columns:")
print(df)
```

```
After renaming columns:
   FRUITS  QUANTITY  PRICE
0   Mango        40   80.0
1   Apple        20  100.0
2  Banana        25   50.0
3  Orange        10   70.0
```

Melting the DataFrame

- The pd.melt(df) function transforms the DataFrame from a wide format to a long format.

```
# Melting (gather columns into rows)
print("\nMelted DataFrame:")
print(pd.melt(df))
```

```
Melted DataFrame:
    variable   value
0     FRUITS   Mango
1     FRUITS   Apple
2     FRUITS  Banana
3     FRUITS  Orange
4   QUANTITY      40
5   QUANTITY      20
6   QUANTITY      25
7   QUANTITY      10
8      PRICE    80.0
9      PRICE   100.0
10     PRICE    50.0
11     PRICE    70.0
```

Creating a Pivot Table

- The df.pivot(...) operation reshapes the DataFrame from long to wide format

```
# Pivot table
pivot = df.pivot(columns='FRUITS', values=['PRICE', 'QUANTITY'])
print("\nPivot Table:")
print(pivot)
```

```
Pivot Table:
        PRICE                        QUANTITY
FRUITS  Apple Banana Mango Orange    Apple Banana Mango Orange
0         NaN    NaN  80.0    NaN      NaN    NaN  40.0    NaN
1       100.0    NaN   NaN    NaN     20.0    NaN   NaN    NaN
2         NaN   50.0   NaN    NaN      NaN   25.0   NaN    NaN
3         NaN    NaN   NaN   70.0      NaN    NaN   NaN   10.0
```

Dropping Rows and Columns

- The df.drop(...) operation can be used to drop rows and columns.

```
# Dropping columns
df1 = df.drop(columns=['QUANTITY'], axis=1)
print("\nAfter dropping QUANTITY column:")
print(df1)

# Dropping rows
df2 = df.drop([1, 3], axis=0)
print("\nAfter dropping rows 1 and 3:")
print(df2)
```

```
After dropping QUANTITY column:
   FRUITS  PRICE
0   Mango   80.0
1   Apple  100.0
2  Banana   50.0
3  Orange   70.0
```

```
After dropping rows 1 and 3:
   FRUITS  QUANTITY  PRICE
0   Mango        40   80.0
2  Banana        25   50.0
```

Creating Joins (Left & Right)

- Two separate DataFrames can be joined using pd.merge(...) method.

```
# Create two DataFrames for merging
df1 = pd.DataFrame({'Fruits': ['Mango', 'Banana', 'Grapes', 'Apple', 'Orange'],
                    'Price': [60, 40, 75, 100, 65]})

df2 = pd.DataFrame({'Fruits': ['Apple', 'Orange', 'Papaya', 'Pineapple', 'Mango'],
                    'Price': [120, 60, 30, 70, 50]})

# Merge examples
print("\nLeft join:")
print(pd.merge(df1, df2, how='left', on='Fruits'))

print("\nRight join:")
print(pd.merge(df1, df2, how='right', on='Fruits'))
```

Left join:

	Fruits	Price_x	Price_y
0	Mango	60	50.0
1	Banana	40	NaN
2	Grapes	75	NaN
3	Apple	100	120.0
4	Orange	65	60.0

Right join:

	Fruits	Price_x	Price_y
0	Apple	100.0	120
1	Orange	65.0	60
2	Papaya	NaN	30
3	Pineapple	NaN	70
4	Mango	60.0	50

Creating Joins (Inner & Outer)

- Two separate DataFrames can be joined using pd.merge(...) method.

```
print("\nInner join:")
print(pd.merge(df1, df2, how='inner', on='Fruits'))

print("\nOuter join:")
print(pd.merge(df1, df2, how='outer', on='Fruits'))
```

```
Inner join:
   Fruits  Price_x  Price_y
0   Mango       60       50
1   Apple      100      120
2  Orange       65       60
```

```
Outer join:
      Fruits  Price_x  Price_y
0      Apple    100.0    120.0
1     Banana     40.0      NaN
2     Grapes     75.0      NaN
3      Mango     60.0     50.0
4     Orange     65.0     60.0
5     Papaya      NaN     30.0
6  Pineapple      NaN     70.0
```

Creating Joins (Inner & Outer)

- Two separate DataFrames can be joined using pd.merge(...) method.

```
print("\nInner join:")
print(pd.merge(df1, df2, how='inner', on='Fruits'))

print("\nOuter join:")
print(pd.merge(df1, df2, how='outer', on='Fruits'))
```

```
Inner join:
   Fruits  Price_x  Price_y
0   Mango       60       50
1   Apple      100      120
2  Orange       65       60
```

```
Outer join:
      Fruits  Price_x  Price_y
0      Apple    100.0    120.0
1     Banana     40.0      NaN
2     Grapes     75.0      NaN
3      Mango     60.0     50.0
4     Orange     65.0     60.0
5     Papaya      NaN     30.0
6  Pineapple      NaN     70.0
```

Concatenation (Row and Column)

- pd.concat() function to combine DataFrames in two ways (i.e. Row-wise and Column-wise).

```
# Concatenation
print("\nRow-wise concatenation:")
print(pd.concat([df, df1], axis=0, ignore_index=True))

print("\nColumn-wise concatenation:")
discount = pd.DataFrame({'DISCOUNT': [5, 7, 10, 8]})
print(pd.concat([df, discount], axis=1))
```

```
Row-wise concatenation:
   FRUITS  QUANTITY  PRICE  Fruits  Price
0   Mango      40.0   80.0     NaN    NaN
1   Apple      20.0  100.0     NaN    NaN
2  Banana      25.0   50.0     NaN    NaN
3  Orange      10.0   70.0     NaN    NaN
4     NaN       NaN    NaN   Mango   60.0
5     NaN       NaN    NaN  Banana   40.0
6     NaN       NaN    NaN  Grapes   75.0
7     NaN       NaN    NaN   Apple  100.0
8     NaN       NaN    NaN  Orange   65.0
```

```
Column-wise concatenation:
   FRUITS  QUANTITY  PRICE  DISCOUNT
0   Mango        40   80.0         5
1   Apple        20  100.0         7
2  Banana        25   50.0        10
3  Orange        10   70.0         8
```

Basic Statistical Summary

- We can perform basic descriptive and categorical analysis using Pandas.

```
# Basic statistics
print("\nDescriptive statistics:")
print(df.describe())
```

```
Descriptive statistics:
       QUANTITY      PRICE
count      4.00    4.00000
mean      23.75   75.00000
std       12.50   20.81666
min       10.00   50.00000
25%       17.50   65.00000
50%       22.50   75.00000
75%       28.75   85.00000
max       40.00  100.00000
```

Basic Statistical Summary

- We can perform basic descriptive and categorical analysis using Pandas.

```
# For object columns
print("\nDescriptive stats for object columns:")
print(df.describe(include=['O']))
```

```
Descriptive stats for object columns:
       FRUITS
count       4
unique      4
top     Mango
freq        1
```

Basic Statistical Summary

- We can perform basic descriptive and categorical analysis using Pandas.

```
# Unique values and counts
print("\nUnique FRUITS:", df.FRUITS.unique())
print("\nValue counts for FRUITS:")
print(df.FRUITS.value_counts())
```

```
Unique FRUITS: ['Mango' 'Apple' 'Banana' 'Orange']

Value counts for FRUITS:
FRUITS
Mango     1
Apple     1
Banana    1
Orange    1
Name: count, dtype: int64
```

Aggregation and Statistical Metrics

- Various aggregation and statistical methods that can be applied to a DataFrame.

```
# Sum and cumulative sum
print("\nSum of PRICE:", df['PRICE'].sum())
print("Cumulative sum of PRICE:")
print(df['PRICE'].cumsum())
```

```
Sum of PRICE: 300.0
Cumulative sum of PRICE:
0     80.0
1    180.0
2    230.0
3    300.0
Name: PRICE, dtype: float64
```

Aggregation and Statistical Metrics

- Various aggregation and statistical methods that can be applied to a DataFrame.

```
# Min, max, mean
print("\nMin PRICE:", df['PRICE'].min())
print("Max PRICE:", df['PRICE'].max())
print("Mean PRICE:", df['PRICE'].mean())
```

```
Min PRICE: 50.0
Max PRICE: 100.0
Mean PRICE: 75.0
```

Aggregation and Statistical Metrics

- Various aggregation and statistical methods that can be applied to a DataFrame.

```
# Median, variance, std
print("\nMedian PRICE:", df['PRICE'].median())
print("Variance of PRICE:", df['PRICE'].var())
print("Std deviation of PRICE:", df['PRICE'].std())

# Quantiles
print("\nQuantiles of PRICE:")
print(df['PRICE'].quantile([0, 0.25, 0.5, 0.75, 1]))
```

```
Median PRICE: 75.0
Variance of PRICE: 433.3333333333333
Std deviation of PRICE: 20.816659994661325

Quantiles of PRICE:
0.00     50.0
0.25     65.0
0.50     75.0
0.75     85.0
1.00    100.0
Name: PRICE, dtype: float64
```

Applying Custom Functions

- A user-defined function can be applied to each column using df.apply(...).

```python
# Custom function
def summation(col):
    if col.dtypes != 'int64':
        return col.count()
    else:
        return col.sum()

print("\nApplying custom function:")
print(df.apply(summation))

# Covariance and correlation
print("\nCovariance matrix:")
print(df.cov(numeric_only=True))

print("\nCorrelation matrix:")
print(df.corr(numeric_only=True))
```

```
Applying custom function:
FRUITS      4
QUANTITY    4
PRICE       4
dtype: int64

Covariance matrix:
            QUANTITY       PRICE
QUANTITY  156.250000    8.333333
PRICE       8.333333  433.333333

Correlation matrix:
          QUANTITY     PRICE
QUANTITY  1.000000  0.032026
PRICE     0.032026  1.000000
```

Handling Missing Data

- Pandas provides different methods to handle the missing data as per requirements.

```
# Create DataFrame with missing values
df_missing = pd.DataFrame({'A': [1, 2, np.nan, 4],
                           'B': [5, np.nan, np.nan, 8],
                           'C': [9, 10, 11, 12]})

# Detect missing values
print("\nMissing values:")
print(df_missing.isnull())

print("\nCount of missing values:")
print(df_missing.isnull().sum())

# Fill missing values
print("\nAfter filling with mean:")
print(df_missing.fillna(df_missing.mean()))

# Drop missing values
print("\nAfter dropping rows with missing values:")
print(df_missing.dropna())
```

```
Missing values:
       A      B      C
0  False  False  False
1  False   True  False
2   True   True  False
3  False  False  False

Count of missing values:
A    1
B    2
C    0
dtype: int64

After filling with mean:
          A    B   C
0  1.000000  5.0   9
1  2.000000  6.5  10
2  2.333333  6.5  11
3  4.000000  8.0  12

After dropping rows with missing values:
     A    B   C
0  1.0  5.0   9
3  4.0  8.0  12
```

Adding new columns

- A new column can be added to a DataFrame.

```
# Add new column
df['Origin'] = ['BH', 'J&K', 'BH', 'MP']
print("\nAfter adding Origin column:")
print(df)
```

```
After adding Origin column:
   FRUITS  QUANTITY  PRICE Origin
0   Mango        40   80.0     BH
1   Apple        20  100.0    J&K
2  Banana        25   50.0     BH
3  Orange        10   70.0     MP
```

Working with dates

- Pandas allows to work with time series data.

```python
# Date range
import pandas as pd
dates = pd.date_range('20230101', periods=4)
df_dates = df.copy()
df_dates['Date'] = dates
print("With dates:\n", df_dates)

# Resampling
df_ts = pd.DataFrame({'Date': dates, 'Value': [10, 20, 30, 40]})
df_ts.set_index('Date', inplace=True)
print("\nResampled:", df_ts.resample('2D').mean())
```

```
With dates:
    FRUITS  QUANTITY  PRICE Origin       Date
0   Mango         40   80.0     BH 2023-01-01
1   Apple         20  100.0    J&K 2023-01-02
2  Banana         25   50.0     BH 2023-01-03
3  Orange         10   70.0     MP 2023-01-04

Resampled:             Value
Date
2023-01-01   15.0
2023-01-03   35.0
```

2. Scikit-learn

BASIC INTRODUCTION

Scikit-learn

- Scikit-learn is one of the most widely used machine learning libraries in Python.
- It provides simple and efficient tools for data mining, data analysis, and machine learning, and is built on top of NumPy, SciPy, and Matplotlib.
- With Scikit-learn, you can:
  - Load and explore datasets.
  - Preprocess data (e.g., scaling, encoding, imputation).
  - Train machine learning models (e.g., classification, regression, clustering).
  - Evaluate model performance with standard metrics.
  - Perform model selection and hyperparameter tuning.
- For getting the idea, we will perform all the steps to use a model.

Loading a Built-in Dataset

- Scikit-learn provides several built-in datasets for experimentation and learning, and the Iris dataset is one of the most commonly used for classification tasks.
- It contains 150 samples of iris flowers from three different species: setosa, versicolor, and virginica.

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)
X.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Splitting the Dataset

- Before training a machine learning model, it is essential to split the dataset into training and test sets. This ensures that we can evaluate how well the model generalizes to unseen data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(120, 4) (30, 4) (120,) (30,)
```

Splitting the Dataset

- Before training a machine learning model, it is essential to split the dataset into training and test sets. This ensures that we can evaluate how well the model generalizes to unseen data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(120, 4) (30, 4) (120,) (30,)
```

Feature Scaling

- Feature scaling is a crucial preprocessing step in machine learning, especially for algorithms that are sensitive to the scale of input features (e.g., logistic regression, SVMs, KNN).

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Training a Logistic Regression Model

- We will use Logistic Regression model for demonstration purpose and train it.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train_scaled, y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

Making Predictions on the Test Set

- After training the logistic regression model, we can use it to make predictions on new, unseen data — in this case, the test set.
- The predictions are based on the learned decision boundary from the training data.

```
y_pred = model.predict(X_test_scaled)
y_pred[:5]
```

```
array([1, 0, 2, 1, 1])
```

Evaluating the Model Performance

- Next, we evaluate the model to understand how well it performed on the test data. Scikit-learn provides several useful functions for this purpose.
- The metrics like precision, recall, f1-score, and support give both a high-level and detailed view of model performance.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Thank You